

Programmierung

Vorlesung, Übung und Tutorium

04 – Datei-Handling und IO

Prof. Dr. Andreas Biesdorf

Tutorium: Nicolas Pfeifer

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Agenda

1

Software Engineering vs.
Programmierung

2

Testen in Python

3

Argumente und
Kommandozeilenbefehle

4

Datei-Handling und IO

5

Algorithmen und Datenstrukturen

6

Such- und Sortierverfahren

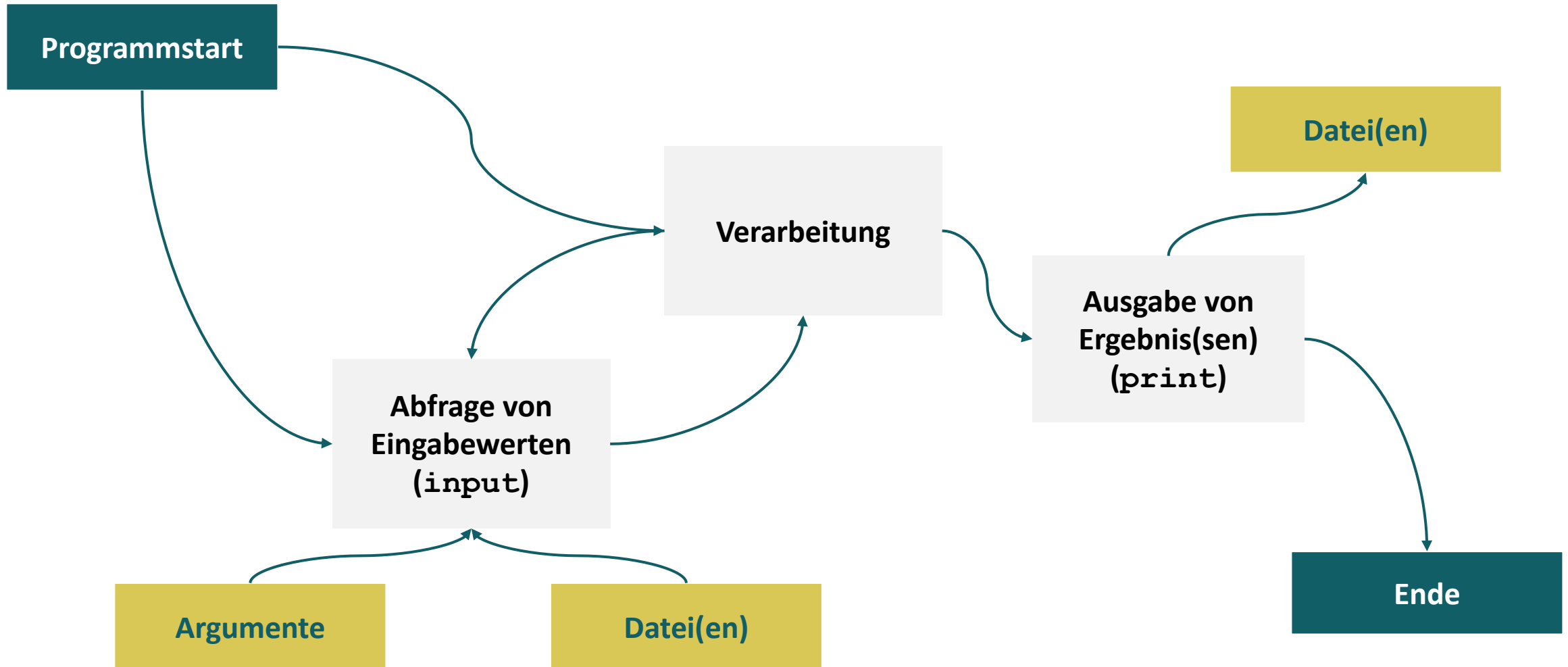


Portfolioprüfung

UNSERE PROGRAMME BISHER

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R



NACHTEILE DER EINGABE MIT `INPUT ()` UND AUSGABE MIT `PRINT ()`

`input ()`

- Schwierigkeiten bei **großen Datenmengen** (z.B. Bilder oder Log-Files)
- **Blockierend** bei Skripten

`print ()`

- Nur **einmalige Ausgabe** der Daten auf der Kommandozeile
- “**Piping**“ der Kommandozeilenausgabe in Datei möglich, aber **umständlich**

4

Argumente und
Kommandozeilenbefehle

5

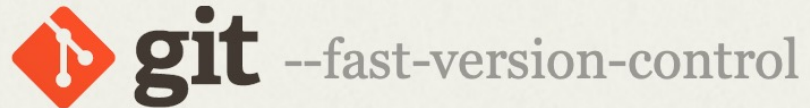
Datei-Handling und IO

FÜR ALLE WINDOWS-NUTZER

<https://git-scm.com/download/win>

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R



Search entire site...

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient **staging areas**, and **multiple workflows**.



Rechte auf Dateien

- Lesen
- Schreiben
- Ausführen

Überprüfung der Rechte durch
den Befehl **ls**

Änderung der Rechte durch den
Befehl **chmod**

Anzeige der Gruppen eines
Nutzers durch **id**

`-rwxrw-r-- 1 nchiapol hep 2450 Sept29 11:52 list1`

ein d hier
markiert ein
Verzeichnis

die 9 Buchstaben zeigen
die Zugriffsrechte dieser
Datei an

Besitzer: nchiapol
Grösse der Datei: 2450
Dateinamen: list1
Gruppe: hep
Erstelldatum: Sept29 11:52

	Eigentümer			Gruppe			Sonstige/Public		
Leserecht	r	-	-	r	-	-	r	-	-
Schreibrecht	-	w	-	-	w	-	-	w	-
Ausführungsrecht	-	-	x	-	-	x	-	-	x

Zeichen	Bedeutung
u	user (Nutzer)
g	group (Gruppe)
o	other (andere)
a	all (alle)
r	read (lesen)
w	write (schreiben und löschen)
x	execute (ausführen bzw. zugreifen für Verzeichnisse)
+	Recht hinzufügen
-	Recht entfernen

Lese- und Schreibrechte für alle:

```
chmod a+rw test.txt
```

Entfernen dieser Rechte:

```
chmod a-rw test.txt
```

- Legen Sie eine **Datei** mit Namen **uebung.txt an** und speichern einen kurzen Text in der Datei
- Welche **Zugriffsrechte** erhält die Datei standardmäßig?
 - Für den **Eigentümer** [user] der Datei?
 - Für die dem Eigentümer assoziierten **Gruppen**?
 - Für alle **anderen Nutzer** des Systems?
- **Entziehen** Sie allen Nutzern die **Lese- und Schreibrechte**
- **Testen** Sie, ob das **Lesen und Schreiben** noch möglich ist
- Geben Sie – ausschließlich – dem Eigentümer das Leserecht wieder zurück

- Erzeugen Sie einen **neuen Ordner** mit dem Namen **testfolder**
- Erzeugen Sie in **diesem Ordner** eine Datei mit Namen **testfile1**
- Erzeugen Sie weiterhin in diesem Ordner **einen Ordner mit Namen testfolder2**
- Erzeugen Sie **in testfolder2** eine Datei mit Namen **testfile2**
- **Entziehen Sie allen Nutzern die Lese- und Schreibrechte auf den Ordner testfolder, lassen aber die Ausführungsrechte bestehen**
 - Frage 1: Ist ein Zugriff auf Datei testfile1 weiterhin möglich?
 - Frage 2: Können Sie sich den Inhalt von testfolder anzeigen lassen?
 - Frage 3: Ist ein Zugriff auf Datei testfile2 möglich?
 - Frage 4: Können Sie sich den Inhalt von testfolder2 anzeigen lassen?

- `cd` – „Change Directory“: wird verwendet, um das aktuelle Arbeitsverzeichnis zu ändern.
- `ls` – „List“: zeigt den Inhalt eines Verzeichnisses an
- `pwd` – „Print Working Directory“: gibt den Pfad des aktuellen Verzeichnisses aus
- `mkdir` – „Make Directory“: erstellt ein neues Verzeichnis
- `rm` – „Remove“: wird verwendet, um Dateien oder Verzeichnisse zu löschen
- `cp` – „Copy“: kopiert Dateien oder Verzeichnisse
- `mv` – „Move“: verschiebt oder benennt Dateien und Verzeichnisse um
- `touch`: erstellt eine neue leere Datei oder aktualisiert das Änderungsdatum einer vorhandenen Datei.
- `echo`: gibt Text oder Variablen auf der Standardausgabe aus.

- Sofern eine Datei interpretierbaren Code enthält, kann man den Interpreter als Kommentar in der ersten Zeile angeben

Beispiel: Python

```
#!/usr/local/bin/python3  
  
print("hello, world!")
```

Beispiel: Bash

```
#!/bin/bash  
  
echo "hello, world!"
```

Achtung: Datei muss ausführbar sein [+x]!

DATEIEN LESEN IN PYTHON

<https://docs.python.org/3/tutorial/inputoutput.html>

- Dateien lassen sich mit dem Befehl `open` öffnen
- Achtung: Python geht vom aktuellen Verzeichnis aus, in dem das Programm aufgerufen wird

```
f = open("morgenstern.txt", "r")  
print(f.read())  
f.close()
```

reines Lesen der Datei

Schließen der Datei

Liest die gesamte Datei ein, man könnte
auch die Anzahl Zeichen angeben, z.B.
`f.read(10)`

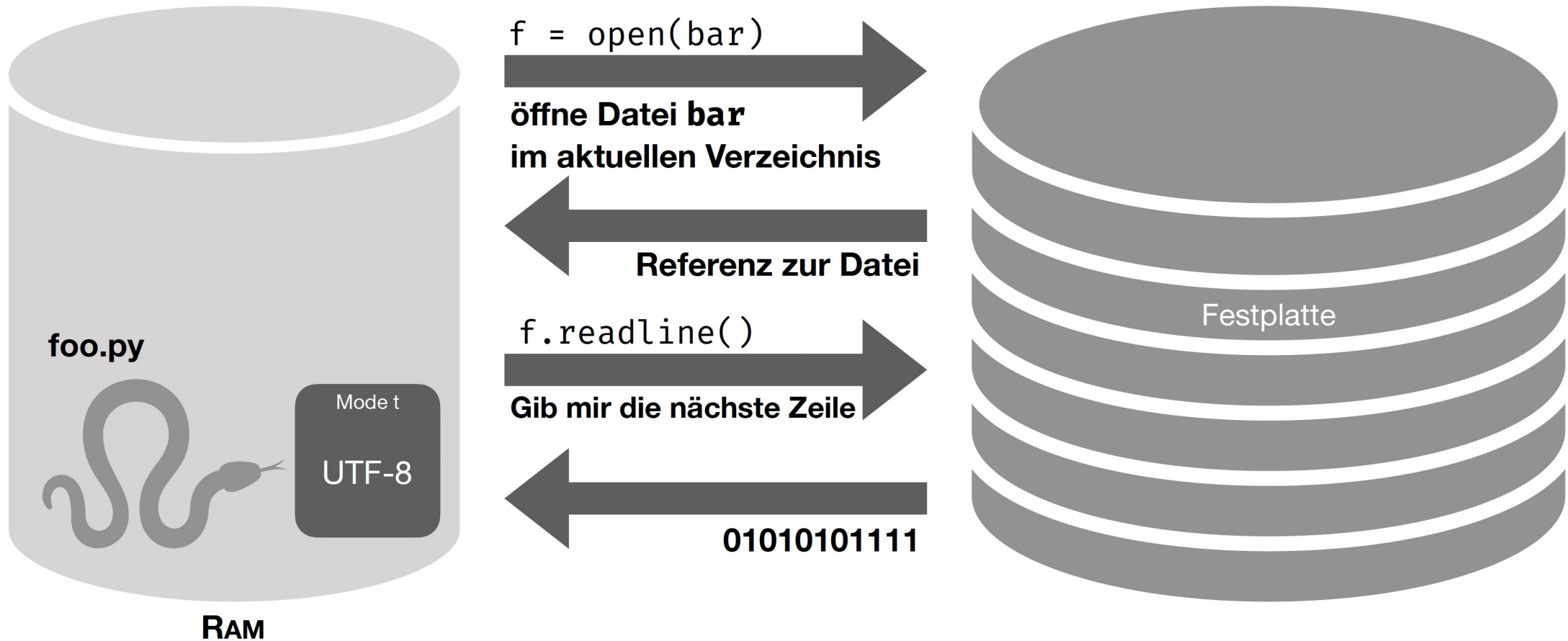
<https://docs.python.org/3/tutorial/inputoutput.html>

Methoden zur Ausgabe der Inhalte der Textdatei

- `read()` – gesamter Inhalt
- `read(x)` – x Zeichen der Datei
- `readline()` – einzelne Zeile
- `readlines()` – Liste aller Zeilen

```
f = open("morgenstern.txt", "r")
zeilennummer = 1
for zeile in f.readlines():
    print(zeilennummer, zeile, end='')
    zeilennummer+=1
f.close()
```

ACHTUNG: DATEIEN SIND STREAMS!



ÜBUNG 3 – WELCHE AUSGABE ERZEUGEN FOLGENDE SNIPPETS?

1

```
f = open("morgenstern.txt", "r")
print(f.read())
print("Kürzere Ausgabe, nur erste Zeile:")
print(f.read(12))
f.close()
```

?

2

```
f = open("morgenstern.txt", "r")
print(f.readline())
print("Stopp!")
print(f.read(4))
print("Stopp!")
print(f.read())
f.close()
```

?

3

```
f = open("morgenstern.txt", "r")
print(f.read(12))
print("Stopp!")
print(f.read())
f.close()
```

?

<https://docs.python.org/3/tutorial/inputoutput.html>

Methoden zum Speichern von Inhalten in einer Textdatei

- `write()` – gesamter Text als ein String

```
f = open("meinWitz.txt", 'w')
f.write("Treffen sich zwei Jäger.\nBeide tot.")
f.close()
```

- `writelines()` – schreiben einer Liste an Strings

```
f = open("meinWitz.txt", 'w')
f.writelines(["Treffen sich zwei Jäger.", "\nBeide tot."])
f.close()
```


DATEIEN LESEN UND SCHREIBEN IN PYTHON

<https://docs.python.org/3/library/functions.html#open>

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	open for exclusive creation, failing if the file already exists
'a'	open for writing, appending to the end of file if it exists
'b'	binary mode
't'	text mode (default)
'+'	open for updating (reading and writing)

```
f = open("meinWitz.txt", "r")  
l = f.readlines()  
f.close()  
print(l)  
l.insert(3, "tralala\n")  
print(l)  
  
f = open("meinWitz.txt", "w")  
f.writelines(l)  
f.close()
```

Idee: Zeilen
auslesen und neue
Zeile einfügen

- `exists()` – Existiert die Datei?
- `getsize()` – Größe der Datei?
- `isabs()` – Absoluter Pfad?
- `isfile()` – Datei?
- `isdir()` – Verzeichnis?
- `realpath()` – absoluter Pfad
- `split()` – Verzeichnis / Datei

```
>>> import os.path
>>> os.path.exists("morgenstern.txt")
True
>>> os.path.getsize("morgenstern.txt")
199
>>> os.path.isabs("./")
False
>>> os.path.isfile("morgenstern.txt")
True
>>> os.path.isdir("morgenstern.txt")
False
>>> os.path.realpath("morgenstern.txt")
'/Users/andreasbiesdorf/Development/hst-wi/prog/Vorlesung/FileIO/morgenstern.txt'
>>> os.path.split("./morgenstern.txt")
('.', 'morgenstern.txt')
```